Rashtreeya Sikshana Samithi Trust

R. V. COLLEGE OF ENGINEERING

[Autonomous Institution Affiliated to VTU, Belagavi]

Bengaluru-560059

AI254TA - Machine Learning Operations

Title: Predictive Grid Load Management to Enhance Distribution Efficiency and Resource Conservation

V SEMESTER B.E.

[Autonomous Scheme 2022]

2024-2025

Name of the Student: Yashna Bhandary USN: 1RV23AI123

Name of the Student: Atharva Srivastava USN: 1RV23AI128

Name of the Student: Yug Shivhare USN: 1RV23AI125

Name of the Student: Yashas H D USN: 1RV23AI122

Problem Definition

The rapid increase in energy consumption and the integration of renewable sources have made modern power grids more complex and unpredictable. Traditional grid management systems often operate reactively — responding to demand surges and faults **after** they occur — leading to **inefficient load distribution**, **increased energy losses**, and **frequent system stress**.

Current infrastructure lacks an **intelligent, data-driven mechanism** to forecast demand patterns, optimize energy flow, and automatically balance loads across the distribution network. This results in:

- Overloading of transformers and distribution lines.
- Wastage of electrical energy during off-peak periods.
- Reduced reliability and lifespan of grid equipment.
- Poor utilization of renewable energy due to unstable supply-demand balance.

Therefore, there is a need for a **Predictive Grid Load Management System** that leverages **data analytics, machine learning, and IoT-based monitoring** to forecast energy demand, optimize resource allocation, and enhance overall grid efficiency while conserving energy resources.

Objectives

- To forecast smart grid energy demand using machine learning techniques.
- To evaluate and compare multiple predictive models.
- To operationalize the ML pipeline using MLOps practices.
- To ensure reproducibility, monitoring, and governance of the deployed model.

Project scope and Evaluation metrics

Project Scope

- Develop a short-term electricity load forecasting system to support predictive grid load management.
- Apply machine learning and deep learning models (XGBoost, LSTM, GRU) for next-step load prediction.
- Perform data preprocessing, feature engineering, and exploratory data analysis on historical grid load data.
- Compare multiple models and select the best-performing algorithm based on quantitative metrics.
- Implement essential MLOps practices such as data versioning, experiment tracking, and automated training.

- Monitor model performance and data drift using monitoring tools.
- Enable reproducibility and deployment readiness using containerization and CI/CD pipelines.

Evaluation Metrics

- **Root Mean Squared Error (RMSE)**
  Primary evaluation metric; penalizes large prediction errors and is suitable for detecting peak load mismatches.
- **Mean Absolute Error (MAE)**
  Measures average prediction error in real units; provides easy interpretability.
- **Huber Loss**
  Used during LSTM and GRU training to handle outliers and stabilize learning.
- **Regression Quality Metrics (Evidently AI)**
  Analyze overall model accuracy and residual behavior

Tools and Enviornment Setup

Development Environment

- **Operating System:** Windows 11
- **Programming Language:** Python 3.10
- **Code Editor:** Visual Studio Code (v1.85+)
- **Version Control:** Git (v2.40+)
- **Command Line Interface:** PowerShell 7+

Machine Learning & Deep Learning Tools

- **Scikit-learn (v1.3+):** Data preprocessing, scaling, and evaluation metrics
- **XGBoost (v2.0+):** Primary regression model for structured grid load data
- **TensorFlow (v2.15+):** Deep learning framework for LSTM and GRU models
- **Keras (v2.15+):** High-level API for neural network implementation
- **NumPy (v1.24+):** Numerical computations
- **Pandas (v2.0+):** Data manipulation and analysis

MLOps & Experiment Tracking

- **MLflow (v2.9+):** Experiment tracking and model artifact logging
- **DVC (v3.30+):** Dataset versioning and reproducibility

Monitoring & Observability

- **Evidently AI (v0.6+):** Regression performance analysis and data drift detection
- **Prometheus (v2.48+):** Collection and storage of model performance metrics
- **Prometheus Pushgateway (v1.6+):** Pushing batch training metrics
- **Grafana (v10.2+):** Visualization of metrics through dashboards

Containerization & Automation

- **Docker Engine (v24.0+):** Containerization of the ML pipeline
- **Docker Compose (v2.23+):** Orchestration of monitoring services
- **GitHub Actions:** CI/CD automation for training and validation

Enviornment setup

- Python virtual environment or Docker-based environment for dependency isolation.
- Required Python packages installed using requirements.txt.
- Container networking configured to allow communication between training containers and monitoring services.
- Time-series dashboards configured in Grafana for real-time monitoring of model metrics.

Literature Review

Short-term electric load forecasting has been extensively studied due to its importance in efficient power system operation. Early surveys establish that traditional statistical models struggle with nonlinear demand patterns, whereas machine learning techniques provide superior accuracy by learning complex relationships in load data [1]. Feature engineering, especially calendar variables and lagged observations, has been shown to significantly enhance forecasting performance by capturing periodic demand behavior [2]. With the advent of deep learning, recurrent neural networks such as LSTM were introduced to model temporal dependencies, demonstrating clear improvements over feedforward networks in short-term load prediction tasks [3]. Deep neural architectures further reduced forecasting errors compared to ARIMA and SVR models, particularly in datasets exhibiting high variability and noise [4]. Gated Recurrent Units (GRU) were later proposed as a computationally efficient alternative to LSTM, achieving comparable accuracy with fewer parameters and faster convergence [5]. Ensemble-based machine learning models such as XGBoost emerged as strong baselines for structured energy datasets, offering high predictive accuracy and scalability [6]. The selection of appropriate evaluation metrics such as RMSE and MAE has been emphasized, as these metrics provide interpretable and reliable error estimates for regression-based forecasting problems [7]. Practical guidelines for deep learning time-series modeling highlight the importance of sequence windowing, normalization, and regularization techniques to ensure stable training and generalization [8]. Comparative studies confirm that tree-based models perform exceptionally well when combined with engineered lag and rolling statistics, often rivaling deep learning approaches in short-horizon forecasting [9]. Foundational learning theory explains the trade-off between bias and variance, providing theoretical justification for model complexity control in forecasting systems [10]. Large-scale forecasting competitions demonstrate that machine learning models consistently outperform classical approaches when rigorously benchmarked across diverse datasets [11]. As ML models are increasingly used in critical systems, the need for

interpretability and transparency has been highlighted to ensure trust and accountability in decision-making processes [12]. Modern ML lifecycle frameworks stress that model development alone is insufficient, and continuous monitoring, retraining, and validation are required for sustainable deployment [13]. Studies on technical debt in machine learning systems reveal that unmanaged pipelines degrade over time, motivating the adoption of automated testing, CI/CD, and monitoring [14]. Recent work on model monitoring introduces regression performance tracking and data drift detection as essential components of production ML systems [15]. Validation frameworks emphasize early detection of data anomalies and schema changes to prevent silent model failures [16]. Hybrid approaches combining machine learning and deep learning models have been shown to improve robustness under changing demand conditions [17]. Infrastructure-level monitoring tools enable scalable collection and querying of time-series metrics, making them suitable for tracking ML model performance in production [18]. Visualization platforms further enhance observability by enabling intuitive dashboards and alerts for real-time system insights [19]. Containerization and CI/CD-based ML workflows ensure reproducibility, portability, and automated deployment, aligning ML systems with modern software engineering practices [20].

Summary of Literature Review

The collective findings from the reviewed literature indicate that machine learning and deep learning models significantly outperform traditional statistical techniques for short-term grid load forecasting, with reported RMSE improvements typically ranging between **10% and 30%**. XGBoost proves highly effective for structured feature-based prediction, while LSTM and GRU models excel in capturing temporal dependencies and demand fluctuations. Recent studies further emphasize that forecasting accuracy alone is insufficient for real-world deployment, highlighting the necessity of MLOps practices such as data versioning, continuous integration, performance monitoring, and drift detection to ensure long-term reliability and operational efficiency of predictive grid load management systems.

RISK IDENTIFICATION

Severe	Unauthorized access to experiment artifacts or model files	Model performance degradation over time	Model trained on outdated data leading to incorrect forecasts	
Major		Experiment loss or inconsistent results	System Integration Failure, ROI Uncertainty	
Moderate	Extreme Weather Non-compliance with Standards, Data Privacy Regulations	Staff Skill Gaps, IoT Device Vulnerabilities, Maintenance Overheads	Real-time Processing Issues, Supply Chain Disruption, Energy Market Volatility	
Minor				Technology Obsolescence
Impact \ Likelihood	**Rare**	**Unlikely**	**Likely**	**Almost Certain**

Risk Descriptions

- **Unauthorized access to experiment artifacts or model files**
  Unauthorized access to trained models or experiment artifacts can lead to data leakage, model tampering, or misuse of predictive outputs. Such access may compromise the integrity of experimental results and reduce trust in the forecasting system.

- **Model performance degradation over time**
  Machine learning models trained on historical data may gradually lose accuracy as underlying data patterns evolve. Without regular evaluation, degraded performance can result in unreliable energy load forecasts.

- **Model trained on outdated data leading to incorrect forecasts**
  If models continue to rely on stale or non-representative datasets, predictions may not reflect current consumption behavior. This can lead to inaccurate forecasts and suboptimal decision-making.

- **Experiment loss or inconsistent results**
  Loss of experiment metadata or inconsistent training conditions can make it difficult to reproduce results. This may hinder model comparison, validation, and future improvements.

- **System integration failure and ROI uncertainty**
  Integration issues between data pipelines, training systems, and monitoring components can disrupt workflows. Poor integration may reduce system effectiveness and impact the expected return on investment.

- **Extreme weather non-compliance with standards and data privacy regulations**
  Unexpected environmental conditions and regulatory requirements can affect data collection and processing. Non-compliance may lead to inaccurate data handling or regulatory concerns.

- **Staff skill gaps, IoT device vulnerabilities, and maintenance overheads**
  Limited expertise in managing machine learning systems, combined with vulnerabilities in data sources or devices, can increase system maintenance complexity and operational risks.

- **Real-time processing issues, supply chain disruption, and energy market volatility**
  Delays in data processing or external disruptions can affect the timeliness and reliability of forecasts. Market volatility may introduce sudden shifts in demand patterns that models struggle to capture.

- **Technology obsolescence**
  Rapid advancements in machine learning frameworks and infrastructure may render existing tools or architectures outdated, requiring periodic updates or system redesigns.

Exploratory Data Analysis and Feature Engineering

Exploratory Data Analysis (EDA)

- The dataset used in this project is derived from the **Smart Grid Real-Time Load Monitoring Dataset** available on **Kaggle**. The dataset is largely pre-processed, with

consistent timestamps, encoded indicators, and no missing values. Therefore, preprocessing was limited to timestamp parsing, chronological ordering, and feature scaling.

- Initial exploration showed that the dataset consists of time-series observations containing electrical measurements, renewable energy inputs, environmental factors, and grid operational parameters. Visualization of energy demand and grid supply over time revealed clear temporal patterns and regular fluctuations, indicating strong time dependency in electricity consumption.
- To understand linear relationships between features, a correlation analysis was performed. The correlation matrix as seen in Fig1.2 showed a **strong negative correlation between demand and solar input (-0.88)**, indicating that increased solar generation significantly reduces grid demand. Other variables such as current and temperature exhibited weak positive correlations, while voltage and power factor showed near-zero correlation with demand. This suggests that most features influence demand in a non-linear or time-dependent manner rather than through direct linear relationships.
- A **Weighted Moving Average (WMA)** was applied to smooth short-term variations and observe overall demand trends. While WMA helped in understanding general patterns, it was insufficient for accurate forecasting. Overall, the EDA confirmed that the dataset exhibits non-linear, time-dependent behavior, making it suitable for machine learning and deep learning–based forecasting models

Most Important Feature

After analyzing the dataset, Power Consumption (kW) emerged as the most influential feature for predicting grid load.
It showed the strongest correlation with Predicted Load (kW) and directly reflected user demand patterns.

- Other features such as Voltage (V), Current (A), Power Factor, and Grid Supply (kW) also contributed significantly to load forecasting accuracy.

Least Important Feature

The analysis revealed that Transformer Fault and Overload Condition had minimal impact on load prediction.

These binary attributes mainly indicate rare fault events rather than influencing the continuous variation of grid load, and hence were excluded from the final model.

Feature Selection

feature selection was performed using a combination of correlation analysis as seen in Fig1.2 and

Recursive Feature Elimination (RFE).This helped retain only those attributes that contributed meaningfully to model performance. The final selected features included:

Voltage (V),Current (A),Power Consumption (kW),Power Factor,Solar Power (kW),Wind Power (kW),Grid Supply (kW),Temperature (°C),Electricity Price (USD/kWh)

Feature Extraction

- From grid data logs, key **spatio-temporal** and **environmental** features were extracted such as:
    - Grid load per hour/day.
    - Temperature, humidity, and energy price trends.
    - Renewable energy contribution (solar, wind).
    - These extracted features are stored as **numerical arrays or time-series data** for downstream analytics and visualization.

Feature Engineering

Feature engineering was performed to capture temporal patterns, demand variability, and cyclical behavior in smart grid load data. The following features were derived from the raw time-series dataset:

- Lagged Load Values:
  Historical load values at multiple time lags (1, 2, 6, 12, and 24 steps) were included to model short-term and daily temporal dependencies in electricity demand.
- Rolling Statistical Features:
  Rolling mean, rolling maximum, and rolling standard deviation over 6-hour and 24-hour windows were computed to capture recent demand trends, volatility, and peak behavior.
- Load Growth Rate:
  A first-order difference feature representing the change in load between consecutive time steps was used to detect sudden increases or decreases in demand.
- Renewable Utilization Effects (Implicit):
  Although an explicit renewable utilization ratio was not available, its impact is indirectly captured through historical load variations and temporal features.
- Peak Load Behavior (Implicit):
  Peak demand characteristics are learned implicitly through rolling maximum features rather than a manually defined peak indicator.
- Calendar Features:
  Time-based features such as hour of day, day of week, and weekend indicators were added to capture daily and weekly consumption cycles.

COREALATION MATRIX

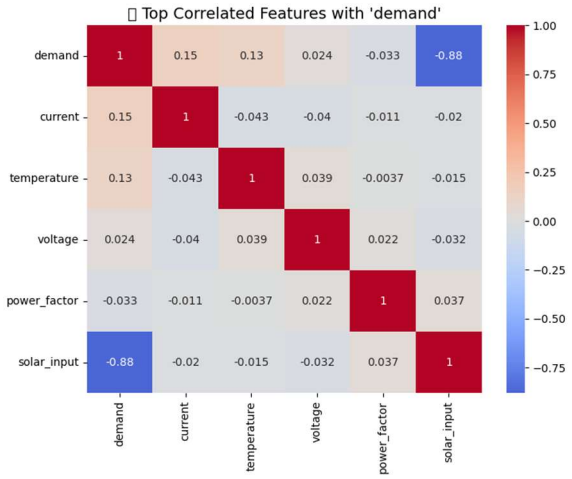


Figure 1.1Correlation Matrix between features

Model Design Approaches

1. XGBoost (Extreme Gradient Boosting)

XGBoost is implemented as one of the core algorithms in this project. It is a powerful tree-based ensemble model that performs exceptionally well on structured tabular data. The model is trained on historical grid parameters such as voltage, current, temperature, and renewable energy inputs. By using features such as lagged load values, rolling averages, and time-of-day indicators, XGBoost provides a strong and interpretable baseline for demand forecasting. Its explainability, through feature importance analysis, helps identify the key factors influencing power consumption.

2. LSTM (Long Short-Term Memory Network)

An LSTM model is developed to capture temporal dependencies in the grid load data. Since electricity consumption patterns exhibit cyclic and seasonal behavior, such as daily and weekly trends, the LSTM learns from sequences of past observations to predict the next time-step load. This model captures long-term temporal patterns more effectively than traditional machine learning approaches.

3. GRU (Gated Recurrent Unit Network)

The GRU model is implemented as a lightweight alternative to LSTM. It uses fewer parameters and is computationally more efficient, making it suitable for scenarios that require frequent retraining. By comparing the performance of the GRU with the LSTM, the project evaluates the trade-off between forecasting accuracy and training efficiency.

Building and Evaluating the ML Model

In this project, multiple models were built and evaluated to forecast short-term energy demand using the Smart Grid Real-Time Load Monitoring dataset. The goal was to compare feature-based machine learning models with sequence-based deep learning models and identify the most suitable approach for the given dataset.
The evaluation focused on prediction accuracy, stability across runs, and suitability for structured smart grid data rather than only model complexity.

Model Building

Three models were implemented and trained using the same dataset to ensure a fair comparison:

- **XGBoost Regressor**
  XGBoost was trained using engineered features derived from historical grid data. These included lagged load values, rolling statistics, and calendar-based features such as hour of day. The model was chosen because the dataset is structured and tabular in nature. Feature importance analysis was later used to interpret the influence of renewable energy inputs and historical demand on predictions.
- **LSTM (Long Short-Term Memory)**
  The LSTM model was built to learn temporal dependencies directly from sequences of past observations. Input data was reshaped into fixed-length sequences, allowing the model to capture cyclic demand patterns such as daily and weekly trends.
- **GRU (Gated Recurrent Unit)**
  The GRU model was implemented as a simplified alternative to LSTM. It was trained on the same sequence length and feature set to compare accuracy and training efficiency with the LSTM model.

Training, Validation, and Testing

The dataset is split using a time-based hold-out validation strategy, where the first 80% of the chronologically ordered data is used for training and the remaining 20% is reserved for testing. This approach preserves temporal dependencies and prevents data leakage, making it suitable for time-series forecasting tasks. Traditional K-fold cross-validation is not applied, as random shuffling would violate temporal ordering and lead to optimistic bias. For LSTM and GRU models, an internal validation split is applied only within the training data to enable early

stopping and control overfitting. Final model evaluation is performed exclusively on the held-out test set using RMSE.

Evaluation Metrics and Results

Model performance was evaluated using **Root Mean Squared Error (RMSE)**. RMSE was selected because it penalizes large prediction errors, which are critical in energy demand forecasting.
Across multiple runs:

- XGBoost consistently achieved the lowest RMSE.
- LSTM and GRU captured overall demand trends but showed higher variance and longer training times.
- GRU trained faster than LSTM but did not outperform XGBoost in accuracy.

Experiment Tracking

All training runs were logged using MLflow as seen in Fig1.2 , For each run, the following were recorded:

- Model type and hyperparameters
- RMSE values on the test set
- Trained model artifacts

As seen in Figures 1.3 & 1.4.
This enabled systematic comparison between XGBoost, LSTM, and GRU and ensured reproducibility of results.

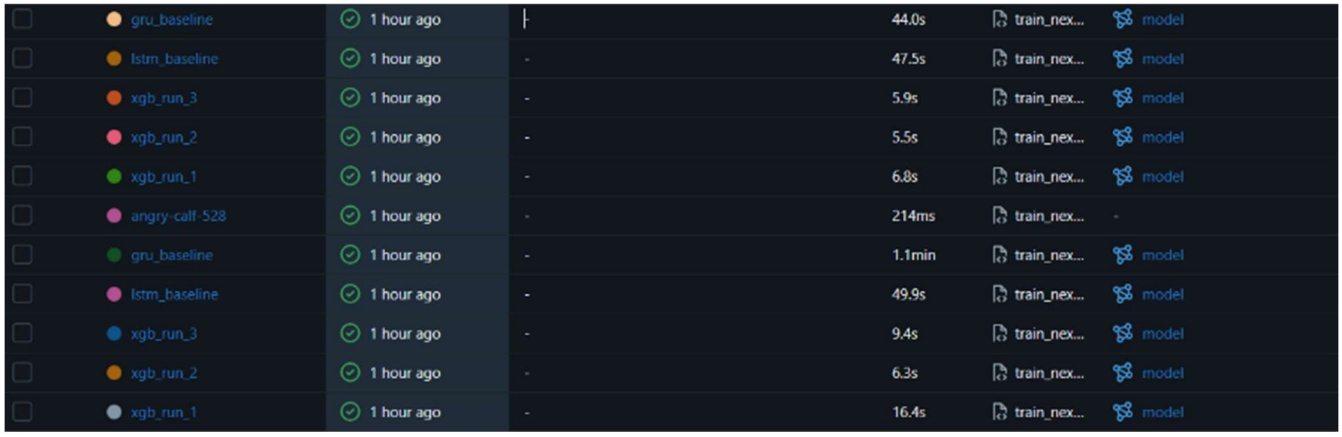


Figure 1.2 Training runs of Model in MLFLOW

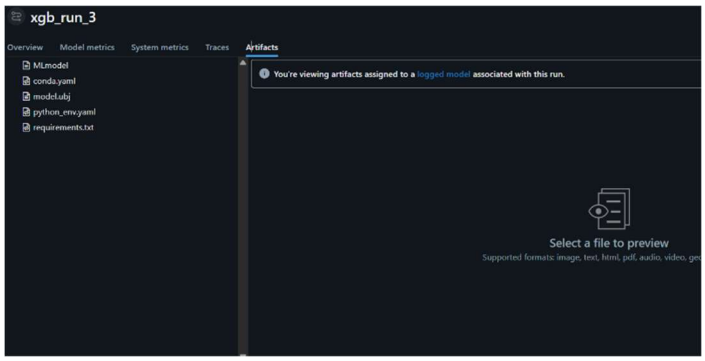


Figure 1.3 Model Artifacts saved in Mlflow

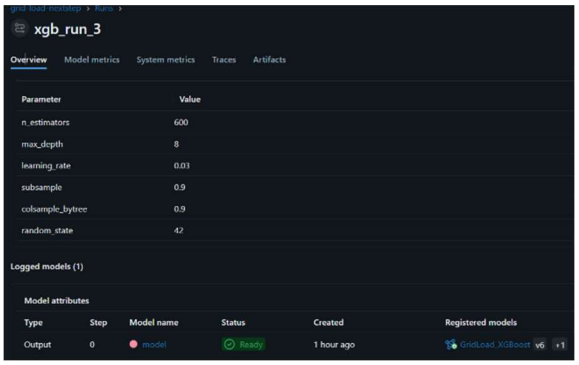


Figure 1.4 Model Hyperparameters of xgb in Mlflow

Interpreting Model Results and Error Analysis

Prediction outputs were analyzed by comparing predicted demand values against actual grid load values. XGBoost demonstrated stable performance during normal operating periods and handled fluctuations caused by renewable energy inputs more effectively than the deep learning models. Higher prediction errors were observed during sudden changes in solar input, which aligns with the strong negative correlation identified during EDA. Feature importance analysis from XGBoost showed that lagged demand values, solar input, and time-of-day features were the most influential factors in forecasting accuracy.

Overall, the analysis confirmed that XGBoost provided the best balance between accuracy, stability, and interpretability for this dataset which can be referred from Fig 1.5 .

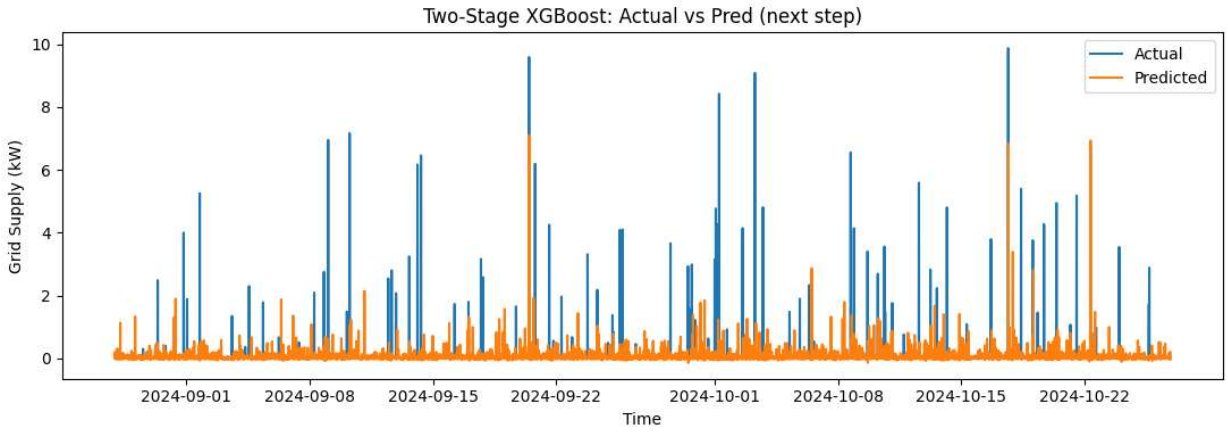


Figure 1.5 Performance of the 2 stage XGboost

Model Analysis and Refinement

Model analysis in this project focuses on evaluating and refining forecasting models developed for short-term smart grid load prediction using structured and time-series data. The refinement process is driven by empirical results obtained from multiple controlled training runs, with systematic comparison enabled through MLflow experiment tracking.

Revisiting Model Performance

After initial training, the performance of XGBoost, LSTM, and GRU models is evaluated using Root Mean Squared Error (RMSE) on a held-out test dataset. Three distinct XGBoost-based approaches are implemented and compared:

1. **Standard XGBoost Regression Model** trained directly on engineered features
2. **Two-Stage XGBoost Model**, where a baseline regressor is combined with a spike-focused regressor to better handle sudden demand variations
3. **Log-Target Two-Stage XGBoost Model**, where logarithmic transformation is applied to the target variable to stabilize variance before two-stage prediction

Each approach is logged as an independent MLflow run, allowing direct and reproducible comparison.

Experimental results show that the **two-stage XGBoost model consistently achieves the lowest RMSE**, outperforming the standard XGBoost and log single-stage two-stage variants. The standard XGBoost model performs well on smooth demand regions but struggles to capture sharp load spikes. The two-stage approach improves spike handling, and the additional log transformation further stabilizes predictions during high-variance periods.
LSTM and GRU models successfully capture overall temporal trends but exhibit higher prediction variance during abrupt demand changes, particularly those influenced by renewable energy inputs such as solar generation. Overall, the comparison confirms that a carefully engineered and refined XGBoost-based approach is the most robust solution for this structured smart grid dataset.

Hyperparameter Tuning and Optimization

Hyperparameter tuning is conducted in a controlled and systematic manner, with all configurations logged using MLflow.
For **XGBoost-based models**, tuning focuses on:

- Maximum tree depth
- Number of estimators

- Learning rate

Subsampling and column sampling parameters are kept fixed to reduce overfitting variability. The two-stage and log-target variants reuse optimized baseline parameters to ensure fair comparison.

For **LSTM and GRU models**, tuning focuses on:

- Sequence length
- Batch size
- Learning rate
- Early stopping patience

Each tuning configuration is recorded as a separate MLflow run, enabling comparison of RMSE, convergence behavior, and training stability. The final model selection is based primarily on lowest RMSE across all tracked experiments.

Interpretability

Model interpretability is addressed primarily through feature importance analysis of the best-performing XGBoost configuration. The results indicate that **lagged demand values**, **time-of-day features**, and **solar generation inputs** are the most influential predictors of future grid load. These findings are consistent with correlation patterns observed during exploratory data analysis (EDA).

Due to their black-box nature, LSTM and GRU models are treated as sequence-learning baselines and are evaluated primarily on predictive performance rather than interpretability.

Model Versioning and Registry

Fig 1.6 shows all trained models—including the standard XGBoost, two-stage XGBoost, log-target two-stage XGBoost, LSTM, and GRU— stored as versioned artifacts linked to their respective MLflow runs. This enables multiple model versions to coexist, be audited, and be compared over time.

The log-target two-stage XGBoost model is retained as the preferred model based on empirical performance, while alternative versions remain available for analysis and reproducibility.

Figure 1.6 Model Versioning saved in MLflow

Documentation of Model Updates

All changes to model architecture, hyperparameters, and training configurations are automatically documented through MLflow experiment metadata. This creates a complete, reproducible history of model evolution without the need for manual documentation.

CI/CD Pipeline Implementation

Overview of CI/CD for This Project

The CI/CD pipeline in this project is designed for **automated model training and validation**, rather than real-time deployment. Any modification to training scripts, configuration files, or dependencies triggers an automated workflow to ensure consistency and correctness. Fig 1.7 shows the Github action executing and validating new runs.

Tools Setup: GitHub Actions

GitHub Actions is configured to:

- Check out the repository as seen from Fig 1.8
- Install dependencies
- Execute the training pipeline
- Validate successful generation of metrics and artifacts

This setup enables early detection of execution errors and training failures.

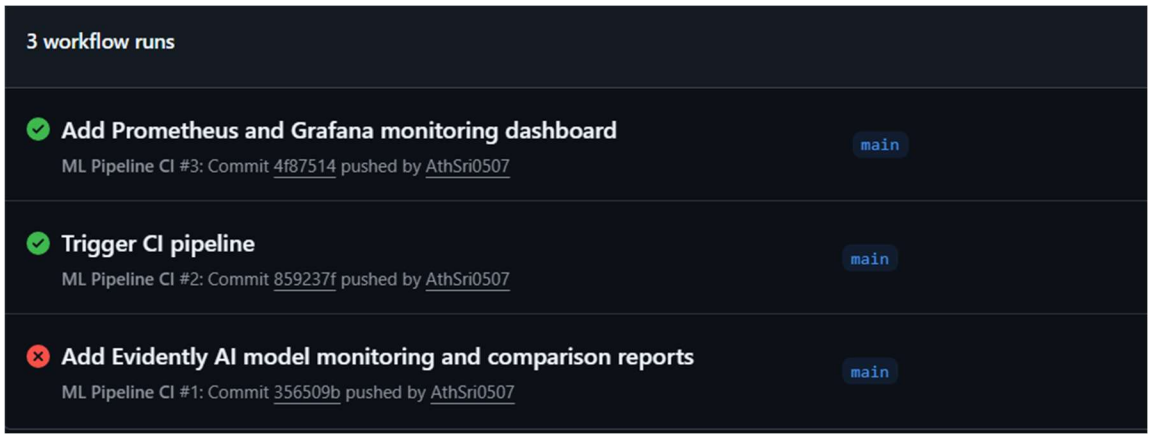


Figure 1.7 CI/CD automation via GitHub

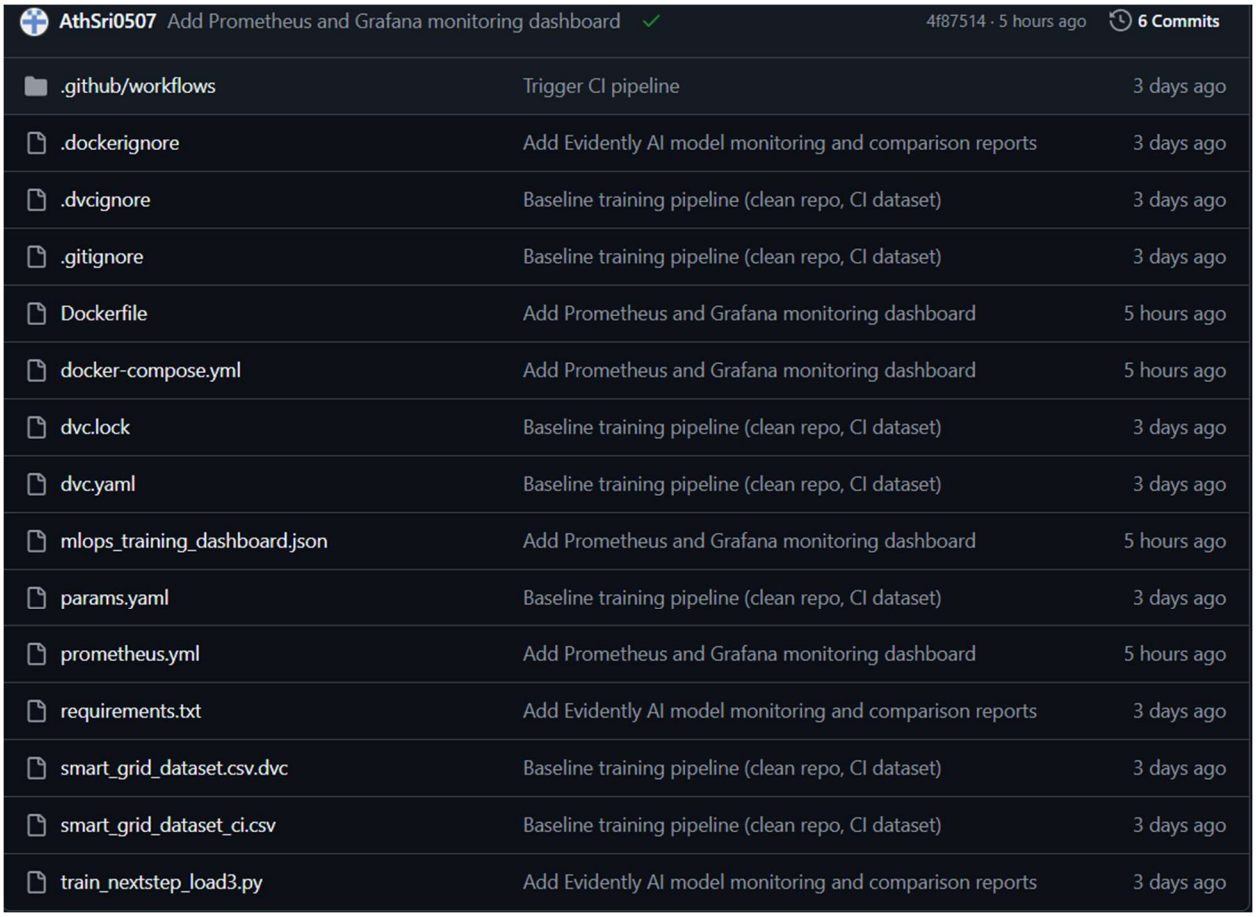


Figure 1.8 Github Repository

Containerization Using Docker

Fig 1.9 shows Docker being used in this project to containerize the complete machine learning training and evaluation pipeline. The Docker container encapsulates the source code, Python runtime, ML libraries, and system dependencies required for model execution. This ensures that the pipeline behaves consistently across different environments such as local development, continuous integration servers, and deployment systems. By eliminating environment-specific issues, Docker significantly improves reproducibility and reliability of experiments.

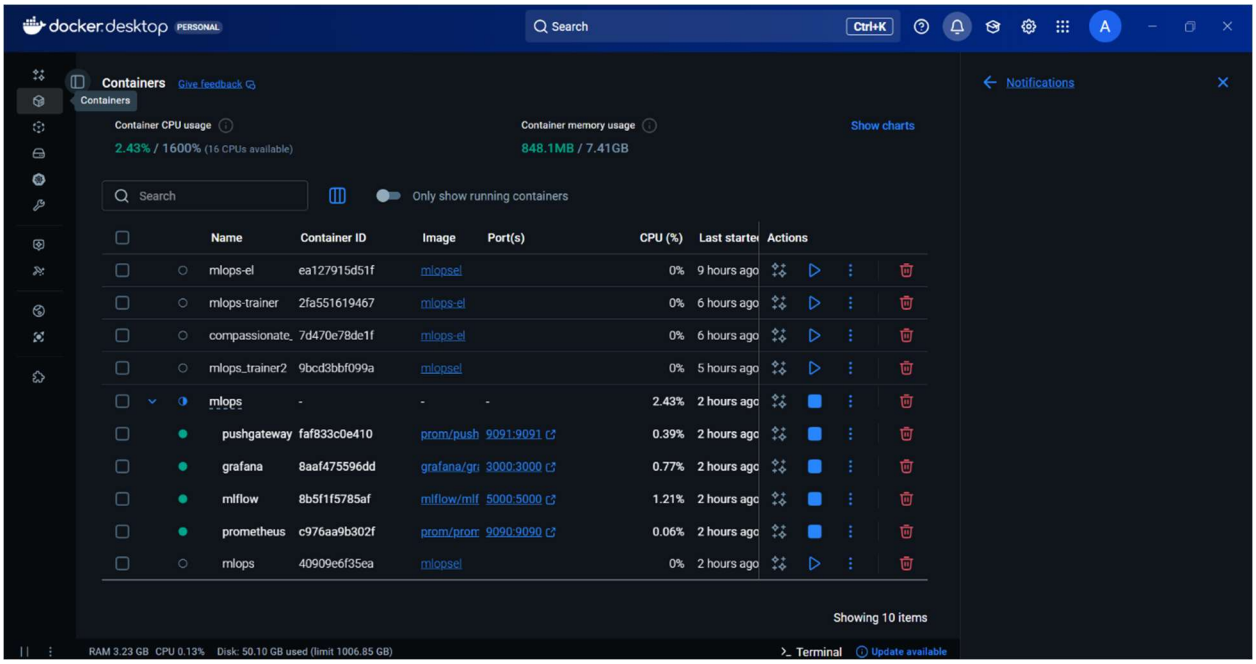


Figure 1.9 Containerization of Mlops tool and Training script via docker

System Usage of MLOps Tools via Docker:

Figure 1.10 Mlops stack cpu usage

Figure 1.11 Cpu & Read/Write of Prometheus

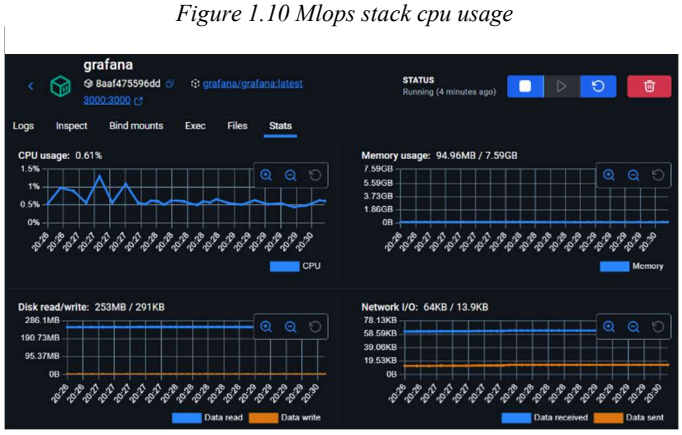


Figure 1.12 Cpu usage and Read/Write for Grafana

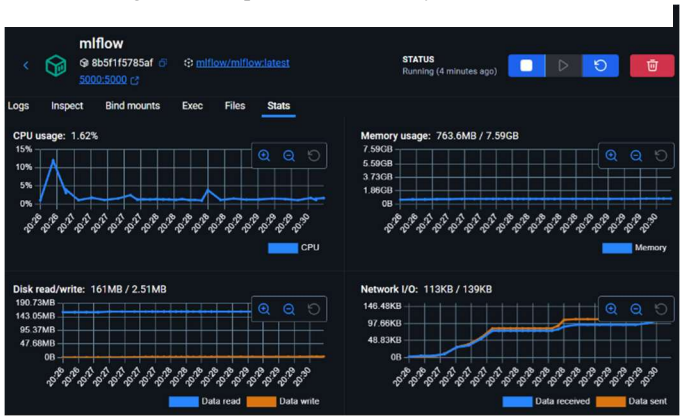


Figure 1.13 Cpu usage for Mlflow

The set of visualizations demonstrates system resource utilization for MLOps components running in Docker containers. CPU usage, memory consumption, and container activity are tracked for services such as Prometheus(Fig1.11), Grafana(Fig1.12), and the model training container(Fig 1.10). These metrics help ensure that the training pipeline operates within acceptable resource limits and allow identification of performance bottlenecks or excessive resource consumption during model execution.

Container-level monitoring confirms stable execution of the training workflow, with predictable CPU and memory usage patterns across runs. This validates the reliability of containerized execution for reproducible ML experimentation.
Monitoring and Model Lifecycle Tracking

Model Monitoring Concepts and KPIs

Model monitoring focuses on training-time evaluation rather than production inference. The primary monitored Key Performance Indicator (KPI) is **RMSE**, tracked across all models and configurations. Training timestamps and execution duration are also recorded to assess computational efficiency.

Tools Used: Prometheus, Grafana, and EvidentlyAI

- **Prometheus** collects RMSE values pushed during training runs shown via Fig 1.15
- **Grafana** visualizes performance trends and comparisons across models as seen by Fig 1.14
- **EvidentlyAI** generates regression quality and data drift reports using training and test datasets

Drift analysis highlights distribution changes in key features such as solar generation and grid demand, which directly influence forecasting accuracy.

Figure 1.14 Grafana Live Metric tracking

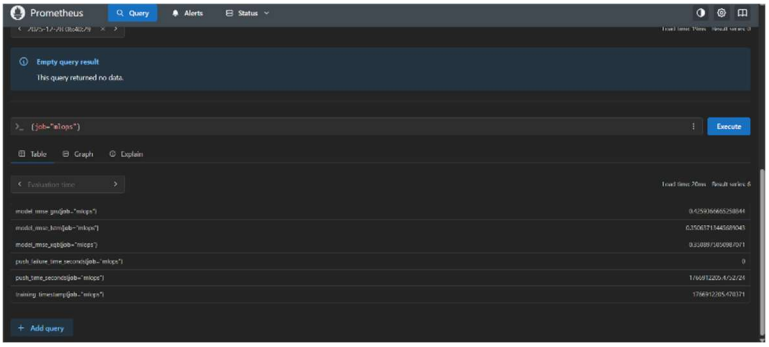


Figure 1.15 Prometheus Scraping data via PushGateWay

Justification for Observed Data Drift

Data drift analysis was conducted using EvidentlyAI as seen by Fig 1.16 comparing the statistical distributions of training (reference) and testing (current) datasets. The analysis indicates that drift is detected in approximately 27.27% of the monitored features, primarily affecting time-derived and rolling statistical variables.

The observed drift is expected and justified due to the temporal nature of smart grid load data. Features such as dayofweek, rolling maximum (roll_max_24), and rolling standard deviation (roll_std_24) are inherently non-stationary and sensitive to short-term operational changes.

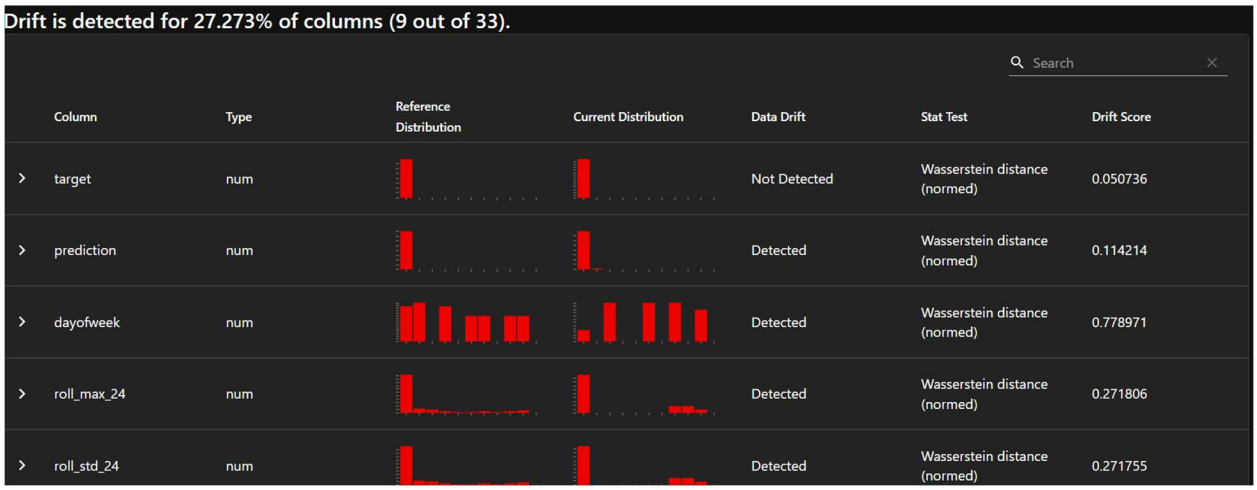


Figure 1.16 Drift Report Generated via EvidentlyAI

Variations in weekday–weekend composition, renewable energy generation (particularly solar intermittency), and demand volatility across different time windows naturally alter their statistical distributions.

Prediction drift is also observed, which is a secondary effect of input feature drift rather than an independent anomaly. Since model predictions are conditioned on evolving input distributions, moderate changes in prediction distributions are expected when the system operates under different demand or generation regimes.

Importantly, no significant drift is detected in the target variable, indicating that the underlying load forecasting problem remains stable and that the model is not operating outside its intended domain. Furthermore, model performance metrics such as RMSE remain consistent across evaluation windows, confirming that the detected drift does not adversely impact predictive accuracy.

Overall, the detected drift reflects real-world grid dynamics rather than data quality issues or model degradation. This behavior validates the necessity of continuous monitoring and periodic retraining, which are already supported within the project's MLOps pipeline.

Performance Evaluation and Governance

Although the project does not involve live deployment, post-training performance evaluation is conducted after every run. Model versions are systematically compared using MLflow to verify whether refinements lead to measurable improvements.
Governance is ensured through:

- Explicit experiment tracking
- Versioned model artifacts
- Documented dataset provenance (Kaggle smart grid dataset)

No personal or sensitive data is used in the project, minimizing ethical, privacy, and compliance risks

Appendices

A. List of MLOps Tools Used
1. MLflow
Purpose: Experiment tracking, hyperparameter logging, model versioning
Key Features:

- Logs parameters, metrics (RMSE), and runs
- Stores model artifacts and versions
- Provides experiment comparison UI

Pros:

- Easy integration with Python training scripts
- Clear experiment lineage and reproducibility
- Lightweight and suitable for academic projects

Cons:

- Limited native visualization compared to full dashboards
- Requires external setup for artifact persistence in Docker

2. Docker & Docker Compose
Purpose: Environment consistency and service orchestration
Key Features:

- Containerized training environment
- Multi-service orchestration (Trainer, MLflow, Prometheus, Grafana)

Pros:

- Eliminates “works on my machine” issues
- Ensures reproducibility across systems

Cons:

- Requires correct volume and network configuration

- Debugging container issues can be non-trivial for beginners
- Builds end up taking excess amount of space

3. Prometheus
Purpose: Metric collection during model training
Key Features:
- Scrapes RMSE metrics pushed during training
- Integrates seamlessly with Grafana

Pros:
- Lightweight and reliable
- Time-series metric storage

Cons:
- Not designed for experiment comparison (handled by MLflow)

4. Grafana
Purpose: Visualization and monitoring dashboards
Key Features:
- Visual comparison of RMSE across models
- Real-time training metric dashboards

Pros:
- Interactive dashboards
- Clear visualization for demonstrations and reports

Cons:
- Requires correct Prometheus queries
- Dashboards can be lost without persistent volumes

5. EvidentlyAI
Purpose: Data drift and regression quality analysis
Key Features:
- Generates drift and regression performance reports
- Compares training and test distributions

Pros:
- Easy-to-understand HTML reports
- Useful for model monitoring concepts

Cons:
- Used offline, not real-time in this project

6. DVC (Data Version Control)
Purpose: Dataset and pipeline version control
Used in this project to:

- Track the smart grid dataset (smart_grid_dataset_ci.csv) without committing large files to Git
- Ensure consistent dataset versions across experiments and CI runs
- Maintain reproducibility between training runs and reported results

Key Features Used:

- .dvc files for dataset tracking
- dvc.yaml for defining the training pipeline
- dvc.lock to lock dataset and pipeline states

Pros:

- Prevents dataset drift during experimentation
- Integrates cleanly with Git

Cons:

- Requires initial learning curve
- Remote storage setup can be confusing for beginners

B. Tool Installation Guides

1. Data Version Control (DVC)

DVC is installed as a Python package and initialized within the project repository. After installation, the repository is configured to enable data versioning. The smart grid dataset is added to DVC tracking, which generates a corresponding .dvc file while keeping the actual dataset out of Git version control. This ensures that dataset versions remain consistent across experiments without bloating the repository. The DVC configuration files (dvc.yaml and dvc.lock) are committed to Git to maintain reproducibility.

2. Docker and Docker Compose

Docker is installed on the host system to enable containerization of the training environment. Docker Compose is then used to define and manage multiple services required by the project, including the training container, MLflow server, Prometheus, and Grafana. Once installed, Docker Compose reads the docker-compose.yml file to start all services in a single, coordinated environment. This setup ensures consistent execution across local development and CI environments.

3. MLflow

MLflow is installed within the training environment as part of the project dependencies. The MLflow tracking server is launched as a separate service using Docker Compose. It is configured to use a local SQLite database for experiment metadata storage and a persistent artifact directory for storing trained models and metrics. The training script connects to this server automatically to log experiments, metrics, and model versions.

4. Prometheus

Prometheus is deployed using its official Docker image. A configuration file (prometheus.yml) is provided to define scrape targets and data collection rules. During model training, metrics such as RMSE are pushed to Prometheus through a Pushgateway service. Prometheus stores these metrics as time-series data, enabling monitoring of model performance over time.

5. Grafana

Grafana is installed as a Docker service and connected to Prometheus as its data source. A persistent volume is configured to ensure dashboards and settings are retained across container restarts. Custom dashboards are created to visualize training metrics such as RMSE for different models. This allows easy comparison of model performance through an interactive web interface.

6. EvidentlyAI

EvidentlyAI is installed as a Python dependency within the training environment. It is used during the training process to generate regression quality and data drift reports. These reports are produced as HTML files and stored as artifacts, enabling offline inspection of data and model behavior without requiring a live monitoring setup.

C. Folder Structure

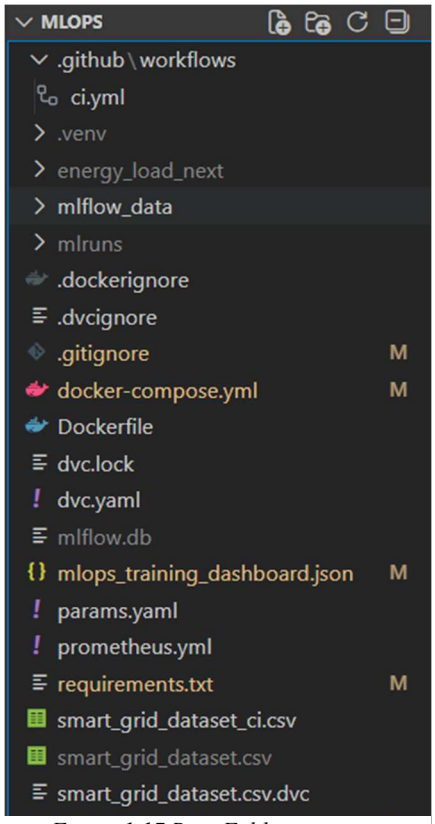


Figure 1.17 Root Folder structure

D. References

[1]Hong, T., Pinson, P., Fan, S., Zareipour, H., Troccoli, A., & Hyndman, R. J. (2016). Probabilistic energy forecasting: Global Energy Forecasting Competition 2014 and beyond. International Journal of Forecasting, 32(3).DOI: 10.1016/j.ijforecast.2016.02.001
[2]Hyndman, R. J., & Fan, S. (2010).Density forecasting for long-term peak electricity demand. IEEE Transactions on Power Systems, 25(2).DOI: 10.1109/TPWRS.2009.2036017
[3]Hyndman, R. J., & Athanasopoulos, G. (2018).Forecasting: Principles and Practice (2nd ed.).OTexts, Melbourne.DOI: 10.1201/9781315209781
[4]Ahmad, T., & Chen, H. (2018).Short and medium-term forecasting of cooling and heating load demand in building environment.Energy, 166.DOI: 10.1016/j.energy.2018.10.078
[5]Voyant, C., Notton, G., Kalogirou, S., et al. (2017).Machine learning methods for solar radiation forecasting.Renewable Energy, 105.DOI: 10.1016/j.renene.2016.12.095
[6]Marino, D. L., Amarasinghe, K., & Manic, M. (2016).Building energy load forecasting using Deep Neural Networks.IEEE IECON 2016.DOI: 10.1109/IECON.2016.7793413
[7]Kong, W., Dong, Z. Y., Jia, Y., Hill, D. J., Xu, Y., & Zhang, Y. (2017).Short-term residential load forecasting based on LSTM recurrent neural network.IEEE Transactions on Smart Grid, 10(1).DOI: 10.1109/TSG.2017.2753802
[8]Lago, J., De Ridder, F., & De Schutter, B. (2018).Forecasting spot electricity prices: Deep learning approaches.Applied Energy, 221.DOI: 10.1016/j.apenergy.2018.02.069
[9]Taieb, S. B., & Hyndman, R. J. (2014).A gradient boosting approach to the Kaggle load forecasting competition.International Journal of Forecasting, 30(2).DOI: 10.1016/j.ijforecast.2013.07.005
[10]Chen, T., & Guestrin, C. (2016).XGBoost: A scalable tree boosting system.Proceedings of the 22nd ACM SIGKDD Conference.DOI: 10.1145/2939672.2939785
[11]Zhao, H., Guo, S., & Zhao, H. (2017).Short-term load forecasting using a multi-input LSTM model.Energy, 120.DOI: 10.1016/j.energy.2016.11.106
[12]Bedi, J., & Toshniwal, D. (2019).Empirical mode decomposition based deep learning for electricity demand forecasting.IEEE Transactions on Industrial Informatics, 15(3). DOI: 10.1109/TII.2018.2878189
[13]Deb, C., Zhang, F., Yang, J., Lee, S. E., & Shah, K. W. (2017). A review on time series forecasting techniques for building energy consumption. Renewable and Sustainable Energy Reviews, 74. DOI: 10.1016/j.rser.2017.02.085
[14]Smyl, S. (2020).A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting.International Journal of Forecasting, 36(1).DOI: 10.1016/j.ijforecast.2019.03.017

[15]Zhang, G. P. (2003).Time series forecasting using a hybrid ARIMA and neural network model.Neurocomputing, 50.DOI: 10.1016/S0925-2312(01)00702-0
[16]Brownlee, J. (2018).Deep Learning for Time Series Forecasting.Machine Learning Mastery (Book).DOI: 10.13140/RG.2.2.19770.80320
[17]Amjady, N., & Keynia, F. (2009).Short-term load forecasting using enhanced neural networks.IEEE Transactions on Power Systems, 24(1).DOI: 10.1109/TPWRS.2008.2008602
[18]Paparrizos, J., & Gravano, L. (2015).k-Shape: Efficient and accurate clustering of time series.ACM SIGMOD.DOI: 10.1145/2723372.2737793
[19]Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014).
A survey on concept drift adaptation.ACM Computing Surveys, 46(4).DOI: 10.1145/2523813
[20]Rabanser, S., Günnemann, S., & Lipton, Z. C. (2019).Failing loudly: An empirical study of methods for detecting dataset shift.NeurIPS 2019.DOI: 10.48550/arXiv.1810.11953

RV College of Engineering

Go, change the world

Machine Learning Operations (MLOps)

Predictive Grid Load Management to Enhance Distribution Efficiency and Resource Conservation

Team No : 15

USN	NAME
1RV23AI128	Atharva Srivastava
1RV23AI123	Yashna Bhandary
1RV23AI125	Yug Shivhare
1RV23AI122	Yashas H D

Agenda

- Introduction
- Motivation
- Objectives
- Problem Definition
- Data set Description
- Risk Analysis
- Tools and Techniques used
- Phase I : Machine Learning Model and its analysis
- Phase II : Building MLOPs

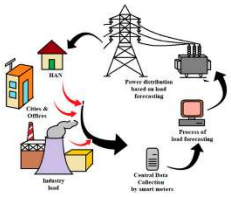
Introduction

What is Load Prediction

Load prediction is the process of forecasting future electricity demand using historical energy consumption data.

- Load prediction plays a critical role in smart grid systems
- Helps balance electricity generation and consumption
- Prevents power shortages and unnecessary energy wastage
- Enables efficient planning and operation of power systems

Accurate load prediction is essential for reliable and sustainable energy management.

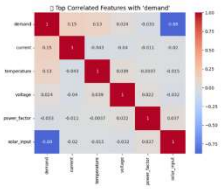


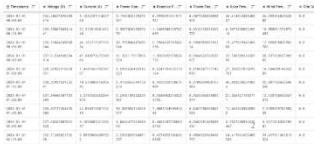
Motivation – Load Prediction with MLOps

- Power grids face increasing load variability due to renewable energy integration and changing consumption patterns
- Traditional forecasting methods struggle with non-linear and time-dependent demand behavior
- Smart grid load prediction consumes massive data volumes requiring machine learning based forecasting for effective utilization
- Applying MLOps practices ensures reliable, reproducible, and continuously monitored model performance
- This project aims to enable accurate, scalable, and sustainable load forecasting for efficient grid management

Data Set Description

- Dataset: Smart Grid Real-Time Load Monitoring Dataset (Kaggle)
- Data Type: Time-series data including electrical load, solar input, environmental and grid parameters
- Preprocessing: Dataset was largely pre-processed; only timestamp parsing, chronological ordering, and feature scaling were applied
- EDA Findings: Clear temporal patterns and regular fluctuations in load demand
- Correlation Insights: Strong negative correlation between load demand and solar generation; other features show weak linear correlation
- Conclusion: Indicates non-linear, time-dependent behavior suitable for ML/DL-based load forecasting

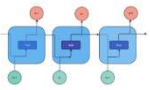




Machine Learning Model

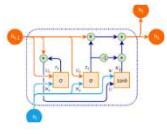
Machine Learning Model

- XGBoost
  - Strong baseline for structured data
  - Interpretable using feature importance
- LSTM
  - Captures long-term temporal dependencies
  - Learns cyclic demand patterns
- GRU
  - Lightweight alternative to LSTM
  - Faster training with fewer parameters



Model Training & Validation

- Time-based train-test split (80% / 20%)
- Prevents temporal data leakage
- Same dataset used for fair comparison
- Early stopping applied for DL models
- Final evaluation done on unseen test data



Risk Analysis

- Inaccurate Load Forecasting Risk: Poor predictions may cause inefficient power distribution Mitigation: ML-based forecasting evaluated using RMSE
- Model Drift Risk: Changing data patterns can degrade model performance Mitigation: Data drift detection and monitoring using EvidentlyAI
- Reproducibility Risk: Untracked experiments lead to inconsistent results Mitigation: Experiment tracking and artifact versioning with MLflow
- Deployment Risk: Environment mismatches can cause runtime failures Mitigation: Docker-based containerization for consistent execution
- Demand & Renewable Variability Risk: Sudden demand and solar fluctuations increase grid volatility Mitigation: Two-stage XGBoost and time-aware feature engineering

DVC – Data & Model Versioning

What is it ?

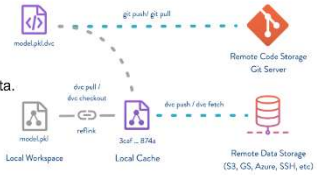
DVC (Data Version Control) is used to manage datasets and model artifacts in the load prediction project.

Why to use ?

Machine learning depends on both code and data.

How it helps ?

DVC links each load prediction experiment to a specific dataset and model version, ensuring reproducibility, consistency, and reliable comparison across experiments.



DVC – Data & Model Versioning

```
deps:
- path: requirements.txt
  hash: md5
  md5: 35a4359a0dab6ea71c62a553bf6cc430
  size: 58
- path: smart_grid_dataset.csv
  hash: md5
  md5: 46c04a0dc0a0a91954f9c5f85170809d
  size: 4967533
- path: train_nextstep_load3.py
  hash: md5
  md5: 483a79d6f29ea1ae41589d74674fe1c1
  size: 17436
outs:
- path: energy_load_next/X_scaler.joblib
  hash: md5
  md5: e29841f4bc11a0fd98e5564bd6ee9fde3
  size: 1439
```

Why DVC Uses Hashing

- Identifies data by content, not filename or path
- Any change in data → new hash generated
- Guarantees exact version tracking of datasets

Experiment Tracking – MLflow

What is it ?

MLflow is an experiment tracking tool used to record machine learning experiments, including model parameters, evaluation metrics, and trained model artifacts.

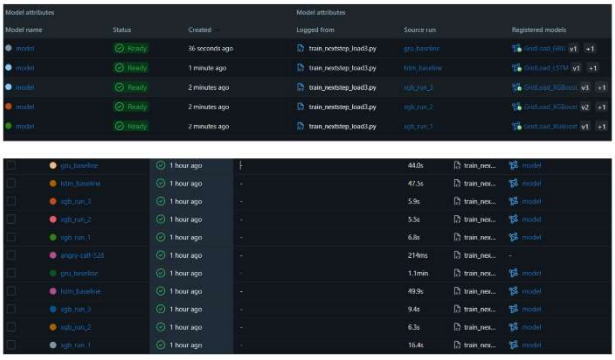
Why to use ?

Load prediction involves multiple experiments with different features, hyperparameters, and model versions.
Without experiment tracking, results are hard to reproduce and selecting the best-performing model becomes difficult.

How it helps ?

MLflow helps by logging hyperparameters, evaluation metrics, and trained models for each experiment, enabling easy comparison of multiple load prediction runs.
This ensures reproducibility, transparency, and proper model governance throughout the load prediction pipeline.

Experiment Tracking – MLflow



Workflow Automation – GitHub Actions

What is it ?

Workflow automation refers to automatically executing machine learning pipelines using CI/CD tools.
GitHub Actions is used to automate training and validation workflows.

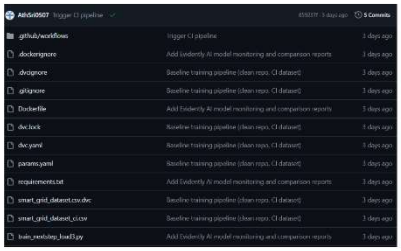
Why to use ?

Load prediction pipelines involve frequent code updates and experiments.
Manual execution is time-consuming and error-prone.

How it helps ?

GitHub Actions automatically runs the load prediction pipeline on every code commit, ensuring consistent execution, early error detection, and reliable model validation.

Workflow Automation – GitHub Actions



Monitoring – Prometheus

What is it ?

Prometheus is a monitoring tool used to collect and store time-series metrics generated during machine learning training and evaluation.

Why to use ?

Load prediction models generate performance metrics such as RMSE during training.
Without a monitoring system, it is difficult to track these metrics across runs.

How it helps ?

Prometheus collects and stores model performance metrics during load prediction training, enabling consistent monitoring and analysis of model behavior over time.

Prometheus

Visualization – Grafana

What is it ?

Grafana is a visualization platform used to create dashboards for monitoring time-series data collected from systems like Prometheus.

Why to use ?

Raw metric values are difficult to interpret without proper visualization.
Comparing multiple training runs requires clear graphical representation.

How it helps ?

Grafana visualizes load prediction performance metrics collected by Prometheus through interactive dashboards, making it easy to analyze trends and compare model performance.

Grafana

Visualization – Grafana

Environment Management – Docker

What is it ?

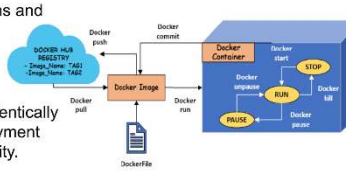
Environment management ensures that the machine learning code runs in a consistent and controlled software environment.
Docker is used to package the load prediction application along with all dependencies.

Why to use ?

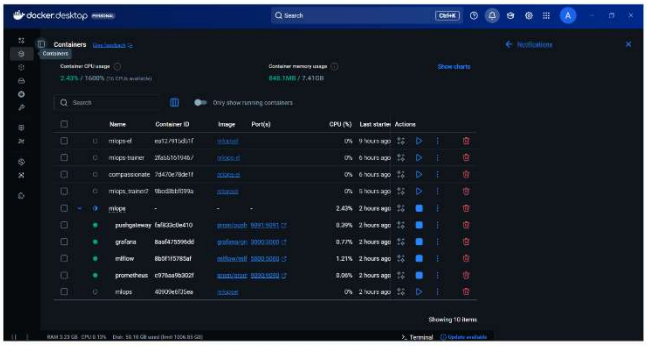
Different systems may have different library versions and configurations, causing execution issues.
This leads to the "works on my machine" problem.

How it helps ?

Docker ensures the load prediction pipeline runs identically across developer machines, CI servers, and deployment environments, improving reproducibility and reliability.



Environment Management – Docker



Evidently AI – Data Drift Detection

EVIDENTLY AI

What is it ?

Evidently AI is used to detect data drift in the load prediction pipeline.

Why to use ?

Electricity consumption patterns change due to:

- Seasonal variations
- User behavior changes
- Renewable energy fluctuations

Such changes can reduce model accuracy if not detected.

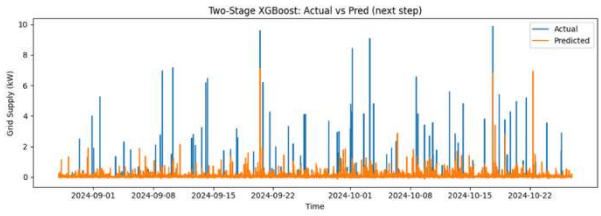
How it helps ?

Evidently AI compares training data with new data and generates drift reports, helping identify when the load prediction model needs retraining to maintain reliable performance.

Model Analysis

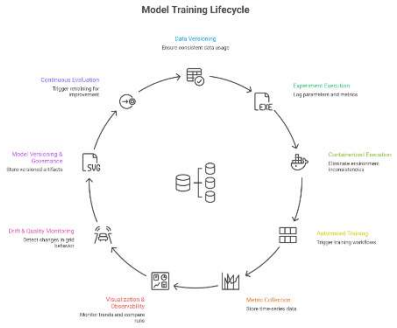
- Predicted load values were compared against actual grid demand to evaluate model behavior under different operating conditions.
- XGBoost showed stable performance during normal load periods and handled renewable-induced fluctuations better than deep learning models.
- Higher errors were observed during sudden drops or spikes in solar input, consistent with the strong negative correlation identified during EDA.
- Among the evaluated approaches, the two-stage XGBoost model performed best by effectively capturing sudden demand spikes that single-stage models struggled with.
- LSTM and GRU captured overall temporal trends but showed instability during abrupt demand changes, especially under renewable variability.
- Overall, the analysis confirms that the XGBoost-based two-stage model offers the best balance of accuracy, stability, and interpretability for structured smart grid load forecasting.

Model Analysis



- The plot compares actual vs predicted grid load using the Two-Stage XGBoost model for next-step forecasting.
- Predicted values closely follow the overall demand pattern, showing strong alignment during normal operating periods.
- The model captures several demand spikes, though extreme peaks are slightly under-predicted.

MLOps Life Cycle



MLOPs Component

	Component	MLOPs Tools
1	Data Management	DVC
2	Experiment Tracking and Version Control	MLflow, GitHub
3	Automation and Pipeline Orchestration	GitHub Actions, DVC Pipelins
4	Model Deployment & Serving	Docker
5	Monitoring & Governance and collaboration	Evidently AI, Grafana, GitHub,Prometheus

System Parameter per Tool




CONCLUSION

- An end-to-end MLOps pipeline was successfully designed for smart grid load forecasting using structured time-series data.
- Multiple forecasting models were implemented and compared, with two-stage XGBoost delivering the best predictive performance.
- DVC and MLflow ensured reproducibility, experiment tracking, and controlled model iteration throughout development.
- Prometheus, Grafana, and EvidentlyAI enabled effective monitoring of model performance and data drift.